

Organização conceitual da libc

A libc tem duas "estruturas": a **conceitual**, definida pelos padrões (ISO C, POSIX), e a **física**, que cada implementação (glibc, musl, newlib) organiza à sua maneira.

1. Organização conceitual (por área funcional)

Cada cabeçalho padrão corresponde a um módulo:

Área	Cabeçalhos	Conteúdo
Strings e memória	<code>string.h</code>	<code>memcpy</code> , <code>strlen</code> , <code>strcmp</code> ...
Classificação de caracteres	<code>ctype.h</code> , <code>wctype.h</code>	<code>isalpha</code> , <code>toupper</code> ...
E/S	<code>stdio.h</code>	<code>FILE</code> , <code>printf</code> , <code>fopen</code> , buffering
Utilitários gerais	<code>stdlib.h</code>	<code>malloc</code> , <code>qsort</code> , <code>atoi</code> , <code>exit</code> , <code>getenv</code>
Matemática	<code>math.h</code> , <code>complex.h</code> , <code>fenv.h</code>	normalmente uma biblioteca separada (libm)
Tempo	<code>time.h</code>	<code>time</code> , <code>strftime</code> , fusos
Sinais e saltos	<code>signal.h</code> , <code>setjmp.h</code>	
Localização	<code>locale.h</code>	
Multibyte e wide char	<code>wchar.h</code> , <code>uchar.h</code>	
Erros e asserções	<code>errno.h</code> , <code>assert.h</code>	
Argumentos variáveis	<code>stdarg.h</code>	
POSIX (extensão)	<code>unistd.h</code> , <code>fcntl.h</code> , <code>sys/*</code> , <code>pthread.h</code> , <code>dirent.h</code> , <code>regex.h</code> ...	

2. Organização em camadas

De cima para baixo:

1. **API pública:** funções do padrão.
2. **Implementação portátil:** C puro, independente de sistema (`strlen`, `qsort`, `printf` formatador, `malloc` sobre um alocador).
3. **Camada dependente de arquitetura:** versões otimizadas em assembly (`memcpy`, `setjmp`, `longjmp`), ABI, startup code (`crt0`).
4. **Camada de syscalls:** o ponto onde a libc encosta no kernel. É a única parte que muda de verdade de um SO para outro.

3. Estrutura de diretórios em implementações reais

musl (a mais legível, ótima para estudar):

```
musl/
├── include/          # cabeçalhos públicos
├── arch/<cpu>/       # código e bits por arquitetura (syscall_arch.h, bits/)
├── crt/              # crt1.o, crti.o, crtn.o (startup)
├── src/
│   ├── string/      stdio/  stdlib/  math/   time/   ctype/
│   ├── locale/      multibyte/ signal/ setjmp/ errno/
│   ├── malloc/      thread/  process/ unistd/ select/
│   ├── network/     passwd/  regex/  env/    exit/
│   └── internal/    # cabeçalhos privados (stdio_impl.h, syscall.h...)
└── ldso/            # carregador dinâmico
```

glibc (mais complexa, com muita camada de override):

```
glibc/
├── string/ stdio-common/ libio/ stdlib/ math/ time/ ctype/ locale/
├── malloc/ signal/ setjmp/ wcsmb/ iconv/ nptl/ (threads)
├── nss/ resolv/ inet/ (nomes e rede)
├── csu/          # startup
├── elf/          # linker dinâmico
├── sysdeps/      # o coração da portabilidade
│   ├── generic/
│   ├── m68k/ x86_64/ arm/ ...      # por arquitetura
│   └── unix/sysv/linux/<arch>/    # por SO + arquitetura
```

O `sysdeps/` funciona por **cascata de busca**: o build procura cada arquivo do mais específico (SO+arquitetura) ao mais genérico, e usa o primeiro que encontrar.

newlib (voltada a sistemas embarcados e SOs próprios):

```
newlib/
├── libc/
│   ├── ctype/ stdio/ stdlib/ string/ time/ locale/ signal/ errno/ ...
│   ├── machine/<cpu>/ # assembly por arquitetura
│   ├── reent/        # versões reentrantes, struct _reent
│   ├── syscalls/     # stubs genéricos
│   └── sys/<os>/      # portes para SOs específicos
└── libm/             # matemática separada
```

4. Por que isso importa se você está portando

O desenho da `newlib` é pensado para esse caso: toda a `libc` depende de um conjunto mínimo de stubs que você implementa (`_write`, `_read`, `_open`, `_close`, `_lseek`, `_fstat`, `_isatty`, `_sbrk`, `_exit`, `_kill`, `_getpid`). Com isso o `printf`, o `malloc` etc. funcionam sem alterar o resto. Na `musl`, o equivalente é trocar a camada de `syscall` e o `arch/`.

Se quiser, posso detalhar uma delas (como o `FILE` e o buffering são organizados internamente, como o `malloc` se encaixa, ou o fluxo de startup `crt0` → `main` → `exit`).

